Told or Enforced: When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Dom Colligan

Abstract

An agent can be kept in accordance with a set of rules in an environment with two methods: state the rules in the agent's prompt and rely on the model to follow them, or have the surrounding layer of software in the harness block undesirable actions, regardless of the model's decision. Real systems do both, such that nobody has measured what the marginal value of one way is given the other. In this experiment, we separate these two methods: the rule told or withheld, and the runtime's enforcement on or off. We ask the question an agent harness designer faces: given the model was told, does enforcement still change behaviour?

In a small, pre-registered, single-runtime study the answer depends on the model. We test the commit boundary: the agent may propose a change to the user's data, but only the user may commit it. Of the runtime's rules we vary, it is the only one that yielded a measurable behavioural contrast; on the rest we saw no meaningful difference to read, so the result rests on this one rule. Told this rule with enforcement off, both capable models (MiniMax-M3 and Llama-3.3-70B) refused on every run of both test tasks, while both weaker models (Qwen3.5-9B and Qwen2.5-7B) committed the change on every run. Capable models enforce the rule themselves; weak ones need the runtime to. We report raw counts rather than a p-value: with two test tasks per side and near-identical repeats, the honest sample size is two, so this is a clear demonstration, not a powered result, and a confounded one at that, since capability tracks model family here.

Two further contributions stand apart from the main finding. First, we release GovernedAgentBench: the set of test scenarios, an automatic grader that is fixed code with no AI or randomness, the exact saved transcripts and grades from our paid experiment, and the precise version of the reference software, identified by its unique code fingerprint. Second, a caution: a test harness that hides a tool's output from the agent can make the agent guess the facts it cannot see, and a grader can then misread those guesses as the agent making things up. An apparent case of this in our own pipeline, where the agent seemed to invent facts whenever it needed one to finish a task, disappeared completely as soon as we showed the agent what its commands actually returned.

1 Introduction

An agent harness is a design surface. Whoever wires up an LLM to tools controls the interaction surrounding it: which tools are exposed, how actions are parsed and dispatched, and what happens when a proposed action would violate a constraint. For any given constraint, that engineer holds exactly two levers. The first is in-context specification (telling): state the rule in the agent's prompt as an in-context contract and rely on the model to respect it. The second is runtime enforcement (enforcing): have the harness block the violation deterministically, regardless of what the model decides. Telling is cheap and moves with the prompt; enforcing requires typed command schemas, dispatch gates, and validation code that must be built, tested, and maintained. A harness designer deciding where to spend engineering effort needs to know when the second lever buys behaviour the first does not.

Existing guardrail evaluations do not answer this, because they run the two levers together. Some toggle enforcement while the full policy sits in the prompt in both conditions [arXiv:2604.15579], so the delta is only measured where the model was already told. Others bundle "injected and enforced" into one switch [arXiv:2602.22302], or compare an enforced arm against a prompt-only arm [arXiv:2603.00822, arXiv:2605.28914], which mixes telling into enforcing. The closest, Mind the GAP [arXiv:2602.16943], varies enforcement directly but never crosses the told and enforced forms of the same rule. Measuring what enforcement adds for a rule the model was already told is nearly absent from prior work, and no located study crosses told-versus-withheld against enforced-versus-off for the same rule, across all four cells, scored on the model's first action.

This paper runs that crossing. For each rule we build a 2×2 experiment: the rule told or withheld, and the

runtime enforcing or off. The quantity we care about is the marginal value of enforcement given the model was told.

We hypothesised that two parameters would decide the marginal contribution of enforcement given in-context specification: model capability and goal conflict. Model capability is the one that carries the result; goal conflict did not, producing no rule-breaking at all (Section 5). On the commit boundary, told the rule with enforcement off, the two capable models complied on every run of both test tasks and the two weak models committed the forbidden change on every run. The split in what enforcement adds is clean: nothing detectable for the capable models, everything for the weak ones. But the honest unit of replication is the task, and there are two per side with near-identical repeats, so we present raw counts and a per-run check of why each model did what it did, not a significance claim. The pattern is stark but small and confounded: capability is entangled with model family (both capable models are non-Qwen, both weak ones Qwen), and once the weakest model is set aside (its failures are partly just being too weak to drive the tools at all), the robust evidence rests on one mid-size model.

A second, independent finding is methodological. Our own harness first produced a spurious result: it handed the agent a file path instead of a tool's actual output, so the blinded agent guessed identifiers it could not see, and the scorer called that fabrication. Restoring the output dissolved the effect entirely (Section 6). We name the failure mode harness blindness.

Contributions. (1) A capability-gated substitution effect: told the commit rule with enforcement off, capable models police themselves on every run and weak ones violate on every run, so enforcement adds nothing detectable above a capability threshold and is the whole barrier below it. We evidence it mechanistically, run by run, not with a p-value, and bound it as a confounded single-runtime case study. (2) The harness-blindness caution, demonstrated by dissolving one such finding. (3) GovernedAgentBench, the instrument that produced the finding above and which we release for reproducibility: a task suite, a deterministic offline scorer, and a git-pinned reference runtime that hold the two levers apart per mechanism. We claim it for its clean, reproducible isolation of telling from enforcing for the same rule, scored on first action, not as a novel design: prior work already reaches the withheld-but-enforced cell, just not that full crossing. The paper claims no scaling law, no injection robustness, and nothing beyond this one runtime.

2 Background and related work

The harness between a model and its tools is now studied as a design surface, but that line optimises it for capability, not governance (Harness-Bench, NLAH, AHE, Meta-Harness [arXiv:2605.27922, 2603.25723, 2604.25850, 2603.28052]; ETCLOVG, OpenReview 3hXEPbG0dh). Enforcement frameworks supply mechanisms without measuring what the model would do unaided (AgentSpec, CaMeL, Progent, Harness-MU [arXiv:2503.18666, 2503.18813, 2504.11703, 2606.21856]), and studies pitting enforcement against text-only governance change behaviour without withholding the rule per condition (ContextCov, Verifier Tax, Mechanical Enforcement, Policy-as-Prompt [arXiv:2603.00822, 2603.19328, 2605.14744, 2509.23994]; and the structural guards AIRGuard, Prompts Don't Protect, Symbolic Guardrails, TRACE, ST-WebAgentBench [arXiv:2605.28914, 2605.18414, 2604.15579, 2606.13174, 2410.06703]). The result is not mere instruction-following, because told-only compliance is not free: capable models finish tasks while breaking latent rules, non-monotonically across families (LogiSafetyBench, SABER, Agent-SafetyBench [arXiv:2601.08196, 2606.01317, 2412.14470]); in-context policy degrades with reasoning complexity (SafePyramid [arXiv:2606.29887]); and rules obeyed while visible are dropped as context compacts (Governance Decay [arXiv:2606.22528]). That checkability is what prior work uses to build evaluations (IFEval, RECAST, VerIF [arXiv:2311.07911, 2505.19030, 2506.09942]), not to predict when an agent complies.

Two priors sit very close. PCAS [arXiv:2602.16708], a policy compiler with a runtime reference monitor, runs a by-design withheld-but-enforced condition: its instrumented arm removes the natural-language policy from the prompt and relies solely on runtime enforcement. But it has no told-and-enforced cell and no neither floor, scores task success after multi-turn corrective recovery rather than on first action, and enforces a single holistic policy per deployment, not a per-mechanism inventory. Mind the GAP [arXiv:2602.16943] is the nearest of all: it varies enforcement directly (unmonitored, observe, enforce, where Enforce hard-blocks forbidden tool calls), so it does contain a rule-absent-but-enforcing cell. Three things still separate it from

our design. Its told rule (a prose safety suffix) and its enforced rule (per-tool RBAC contracts) are different objects, never the same constraint crossed told-against-enforced; it scores the model's attempt rate on intent across multi-turn jailbreak scenarios, not first-action compliance under benign pressure; and its headline "no deterrent" is a power-limited null ($p > 0.27$), not a clean substitution result. We take up the apparent tension with enforcement-helps priors in Section 8.

Others reach part of the crossing. Agent Behavioral Contracts [arXiv:2602.22302] evaluates runtime contracts at scale (1,980 sessions, 7 models) but toggles specification and enforcement together: its conditions "differ only in whether ABC contract rules are injected and enforced" (Table 6), so it never separates the levers and has no off-diagonal cell. We adopt PhantomPolicy's [arXiv:2604.12177] eight-category policy-invisible-violation taxonomy, though its own conditions are a flat, non-crossed set folding telling into enforcement architecture, with no reported enforced-and-told cell. ABSTAIN [arXiv:2606.02965] reaches the closest cell coverage, three of four cells on our specify-by-enforce mapping for one abstention mechanism, but its enforced (Checkpoint) arm "receives the same prompt as the Prompt-Only policy" (Appendix B), realising told-and-enforced rather than withheld-and-enforced, and its unenforced arms are LLM-judged. TeamBench [arXiv:2605.07073] finds prompt-only and sandbox-enforced role separation statistically indistinguishable on pass rate yet 3.6 times more verifier code-edit attempts in the prompt-only arm: convergent evidence for the substitution shape in a different, multi-agent domain where the rule is never withheld, with a standing caution (Section 7) that aggregate parity can mask sub-metric enforcement value.

Isolated withheld-but-enforced measurements exist: PCAS by design, Prompts Don't Protect structurally, and Mind the GAP through its enforce mode. What no located prior offers is a clean, reproducible isolation of the two levers for the same rule across all four cells, scored on first action, per governance mechanism, with a released deterministic offline scorer. GovernedAgentBench provides that instrument; the paper's claims are the empirical finding above and the harness-blindness caution (Section 6), not the novelty of any single design element.

3 The two levers

A harness can make an agent respect a rule two ways: state the rule where the model can read it (telling), or make the violation impossible whatever the model emits (enforcing). Deployed systems pull both at once, so their effects are confounded. This paper's whole manipulation is to decouple them, varying telling and enforcing independently for a single rule.

Telling is the in-context contract. It is not just a policy paragraph but everything the model is shown: the list of commands and their argument shapes, a per-command "safe for the agent to run" flag, and the prose describing the out-of-scope boundary. It is a harness-level lever, separate from anything baked in during training, which we refer to as model disposition.

Enforcing is what the runtime does regardless of the model. Our reference runtime has five governance checks, each of which can be switched off (individually, or all at once):

Check	What it does
schema check	rejects malformed or invented commands before they run
dispatch gate	refuses to run a command not flagged safe for the agent
commit gate	the agent may propose a change to user data, but only the user may commit it
refusal rule	refuses out-of-scope requests, including a zero-tolerance medical boundary
audit check	attaches evidence to recommendations; the scorer later catches fabricated or missing references

A sixth property, transaction integrity, is always on. The commit gate is the one this paper's headline turns

on: the runtime refuses an agent-issued commit and leaves the change merely proposed.

The 2×2. Crossing the two levers, for any one rule, gives four cells:

	Enforcing	Off
Told	A: deployment baseline	B: told, not enforced
Withheld	C: withheld, still enforced	D: neither

Cell A is how systems ship. Cell B isolates self-enforcement: the model knows the rule and nothing stops it. Cell C isolates pure enforcement: the rule is hidden but the runtime still blocks. Cell D is the floor. Three reads carry the information: B vs. D is the effect of telling; C vs. D is the effect of enforcing; and A vs. B is our headline: what enforcement adds once the model was told. If A and B match, enforcement changed no behaviour on that rule; its value is then the guarantee itself, which is real and unconditional (a blocked action simply cannot happen) even where behaviour does not move.

One scoring convention matters. A blocked action returns an error, and that error is itself the rule, told late, so cell C drifts toward cell B after the model's first contact with a block. We therefore score the telling reads on the model's first action, and report multi-turn behaviour separately.

4 GovernedAgentBench

GovernedAgentBench is the instrument that holds the two levers apart. It toggles telling and enforcing independently, per rule, against one fixed runtime, and scores the result deterministically and offline.

The runtime. The instrument is HAI, a non-clinical personal-wellness agent runtime with a typed command interface, pinned as a frozen snapshot; it is a reference implementation, not the contribution or a product, though its author built it in work separate from this paper, which we flag as a same-team caveat. Its no-diagnosis boundary is one of the evaluated rules, not a disclaimer. Two facts matter for reproduction. First, the published v0.2.0 package does not enforce; the dispatch gate landed three days after that version tag, so the measured runtime is pinned by git commit 6c82cd0. Second, before spending anything, a pre-flight check confirmed the runtime actually refuses an agent commit; it passed, so the enforced cells were measured against a runtime verified to enforce, not assumed to.

Tasks. The suite has 39 tasks; the paid run used a concentrated 16-task subset, the rest covered offline. Each task is one labelled cell of a rule's 2×2. For the commit boundary we use two tasks (committing a target and committing an intent), each run four times per cell.

The two axes in code. Enforcing is a runtime setting. Telling is a per-task contract applied at prompt-render time while the runtime is untouched: the withheld prompt strips the rule's facts and prose, and an automatic scan rejects any withheld prompt that still leaks the rule's words. Two caveats attach. First, even withheld, the prompt still lists command names, so the model retains a hint, and B-vs-D and C-vs-D are therefore lower bounds on the effect of telling. Second, a small patch that lets the withheld model still find the commit command was applied after the pre-run canary, so the withheld violation rate carries that provenance and can be re-scored without the patch offline.

Scoring. The scorer is deterministic and offline, with no model calls. We disclose one thing plainly: the harm-check code was edited three times in the day before the run, the last 47 minutes before it, each edit in the direction we hoped for. We verified this does not touch the result: every violation in the headline table is caught by the original, older detection path; re-scoring with the pre-edit scorer is identical on the 2×2, so the finding is invariant to those edits.

The pre-registered run. The design was locked before the run. We disclose two departures. First, the pre-registered primary model was deprecated by the provider mid-cycle; we replaced it with MiniMax-M3, but only after a canary confirmed the replacement reproduced the effect (an outcome-conditioned choice), so MiniMax-M3 is a non-pre-registered addition and its result is exploratory; the pre-registered capable model is Llama-3.3-70B, which shows the same pattern. Second, the commit boundary has two test tasks, not the

three the sample-size analysis assumed, so its pooled cell is eight runs, below our own twelve-run target; the effects are saturated (0/8 vs. 8/8), but the small size is stated, not hidden.

5 Results

We report the four-model capability ladder (Section 5.1); the earlier single-model probes that shaped the design follow as provenance (Section 5.2). One structural fact frames everything. Cell A (told, enforced) and cell C (withheld, enforced) are safe on every run by construction: the runtime blocks the commit whatever the model emits, so all 64 enforced runs score identically. The entire behavioural finding therefore lives in the two unenforced cells, B and D.

5.1 The capability ladder

The run swept four models against the git-pinned runtime, MiniMax-M3 and Llama-3.3-70B (capable), Qwen3.5-9B (near the floor) and Qwen2.5-7B (below it), for a total paid cost of USD 10.44. Scoring the model's first action, the share of runs that stayed safe on the commit boundary was:

Model	A: told, enforced	B: told, off	C: withheld, enforced	D: neither
MiniMax-M3 (capable)	100*	100	100*	0
Llama-3.3-70B (capable)	100*	100	100*	25
Qwen3.5-9B (near floor)	100*	0	100*	12.5
Qwen2.5-7B (below floor)	100*	0	100*	12.5

*Safe by construction; the finding is in columns B and D. Each cell is eight runs (two tasks, four repeats each).

Read column B: told the rule, runtime off. Both capable models stayed safe on all eight runs; both weak models committed the forbidden change on all eight. The marginal value of enforcement given the model was told, cell A minus cell B, is therefore zero for the capable models and the entire barrier for the weak ones. That single column is the paper's finding.

Why each model did it, read run by run. The transcripts show the split directly, and this is the evidence we lean on. In the told-off cell, MiniMax refuses the commit on its own turn and tells the user to run it themselves; Qwen3.5-9B issues the commit command itself, and it executes because the gate is off. The capable model declines an action it is free to take; the weak model takes it. We trust the split at a small sample size because it is not a bare count but a visible mechanism, the same in every run.

What the uncertainty is, by estimation rather than testing. With eight runs a cell the exact (Clopper-Pearson) intervals are wide: a cell at 0/8 safe has interval [0, 37%], and a cell at 8/8 has [63, 100%]. So the weak models' safe rate is bounded at most 37%, and the capable models' at least 63%, but neither is pinned precisely. In particular, the capable side's flat cell A minus cell B is a failure to detect an effect at this size, not proof that none exists: since the 8/8 interval reaches down to 63% and cell A is exactly 100%, an undetected A-minus-B effect as large as about 37 points is still compatible with what we saw. We report it as an upper bound, not an equivalence.

Why we report counts, not a p-value. A significance test is the wrong instrument for this design, for two reasons. First, its assumptions fail: one arm of every enforced contrast is 100% safe by construction rather than a random outcome, and the four repeats of each task are near-identical rather than independent (seven of the eight told-off conditions are byte-for-byte the same across all four repeats), so the shuffling model a p-value is computed under never described the data. Second, the null it would reject, that safe rates

are exactly equal across models, is not a hypothesis anyone held, and any rejection is confounded with model family and task. The number makes the point on its own: treating the eight runs a side as independent gives $p = 0.00016$, but counting the two genuinely independent tasks a side gives $p = 0.33$. We therefore rest the finding on the raw counts, the interval estimates, and the per-run mechanism above, and offer no p-value as evidence.

This is substitution, not the model simply being agreeable. One might read the capable models' flat A-minus-B as models that would comply regardless, needing neither lever. The withheld floor rules that out. With the rule withheld and the runtime off (cell D), the capable models violate too: MiniMax on all eight runs, Llama on six of eight. So telling has a large within-model effect even for a capable model, and its flat A-minus-B is genuine substitution of specification for enforcement, not indifference to both. (The withheld cells depend on the discoverability patch of Section 4 and can be re-scored without it offline.)

Three caveats on the reading. First, the below-floor 7B fails partially rather than totally: its told-off scores are graded (mostly half-violations, some full), so at a lenient threshold that counts only a full violation its column B would read about 75% rather than 0%. The load-bearing cells are unaffected by the threshold, since the capable models at 100% and the near-floor 9B at 0% read the same either way. Second, empty output cannot fake the result: no headline-cell run was scored safe merely because the model emitted nothing, and Llama produces malformed output on roughly 40% of runs yet still reaches 100% in column B through genuine refusals. Third, and most limiting, once the below-floor 7B is set aside (its failures are partly an inability to drive the tools at all) and the capability-versus-family confound is held in view, the whole "told but does not self-enforce" result rests on one mid-size model, Qwen3.5-9B, at eight runs on one rule and one runtime. It is a real, mechanistically verified failure, not yet a multi-model or family-general one.

The other rule we swept is uninformative. The medical-boundary refusal is near-ceiling and carries no result. Its violation detector fires on 7 of 223 runs, all on a single task and a single model; the two tasks written to provoke a medical claim elicit none, even below the floor. We read nothing off it, and note that the near-total silence may reflect genuine boundary-respect or a detector too narrow to catch a violation (Section 7).

5.2 What the earlier one-model probes showed

Before the ladder, a set of single-model probes (Qwen3-235B, three to five runs a cell) shaped the design; we summarise them as provenance. With the runtime off, a told-and-salient model refused three of three while a withheld model committed the violation three of three, establishing that telling and enforcing each move behaviour off the floor. Two ideas we expected to matter did not survive. The rule we predicted a model could not self-check, audit faithfulness, proved self-checkable once the model retrieved the evidence, which is why it collapses into capability rather than standing as a separate factor. And benign pressure to finish the task produced zero fabrication across 40 runs. One qualifier the probes raise and the ladder does not settle: self-enforcement is salience-sensitive. The same told model that refused when the rule was foregrounded dispatched the forbidden command three of three when it met the rule only incidentally. That rests on three runs of one off-roster model and is not independently reconstructable, so we carry it as a caveat, not a result.

6 Harness blindness

Probing prior to the experiment produced and then destroyed an apparent positive result, and the failure mode is general enough to be considered a finding. Early on in this project, the harness did not show the agent its tools' output: where a command's result should have appeared, the agent got a file path instead. It could run a lookup but never read the answer. When a task then needed an identifier to proceed, the agent guessed. Across three probe sets, we found eight cases where it issued commands carrying plausible identifiers it had never seen, including an invented evidence-card id. This looked like a real finding: honest when asked plainly, fabricating when an id was instrumentally required. It was our strongest surviving positive result.

It was an artefact of the harness. We pre-registered a falsification test and re-ran it after fixing the harness to show the agent its command output. Fabrication dropped to zero against the 40-point bar: once the agent

can see the empty lookup, it abstains honestly even when it needed the id to finish.

The lesson: an eval harness that hides tool output can cause the agent to guess, and a scorer reads the guess as fabrication. The number is a fact about the harness, not the model. Genuine constraint-driven fabrication does exist under more adversarial setups [arXiv:2606.14831]; our point is narrower. Before charging an agent with making something up, check that it could see what it is accused of inventing. A related corollary falls straight out of the 2×2: an ablation that toggles enforcement while leaving the policy in the prompt in both conditions [arXiv:2604.15579] measures self-enforcement, not enforcement, and a small delta there bounds nothing about a withheld agent.

7 Limitations and scope

This is a small case study on one runtime. Every model-backed number comes from one runtime, one prompt template, four models, eight runs per headline cell. It is a single-runtime case study by decision: whether the effect generalises beyond this runtime is the highest-value follow-up, and until an independent replication exists, nothing here separates a real phenomenon from a quirk of this instrument.

Capability is confounded with model family (capable = non-Qwen, weak = Qwen), so the ladder cannot fully separate "weaker models need enforcement" from "Qwen models do." The two capable models agreeing across families strengthens the substitution side, but the enforcement-is-load-bearing side, after setting aside the below-floor 7B, rests on Qwen3.5-9B alone. Per-model vendor-recommended sampling also entangles model identity with sampling settings across the ladder, though the per-model 2×2 is computed at one setting and is unaffected.

The saturated cells cut both ways. Because the runtime blocks the commit regardless of the model, cells A and C are 100% by construction; all the behavioural signal lives in B and D, and A-vs-B is a one-sided quantity, not a two-sided contrast. We report it as an upper bound, not an equivalence.

Pre-registration honesty. Beyond the outcome-screened replacement model and the sub-target sample size (Section 4), the withheld prompt still leaks command names, and the medical-boundary detector shares ground truth with the runtime. TeamBench's caution applies to us too: our scorer sees whether the specific guarded violation occurs, but an enforcement effect that shifted some finer channel while leaving the tracked violation unchanged would go undetected at this size.

8 Discussion: what enforcement is still for

The result relocates enforcement's value; it does not remove it. Enforcement keeps its unconditional guarantee: a blocked action cannot happen, whatever the model does or is prompted, and that guarantee is untouched by any behavioural number, including a 100%-refusal rate at tiny n. Beyond it, enforcement contributes behaviourally wherever a model cannot self-enforce: below the capability floor, where the failure is the model being too weak to drive the tools rather than disobedience; when the rule was never told, the floor cell where the runtime is the only barrier and real deployments accumulate such rules through drift and context compaction [arXiv:2606.22528]; when a told rule is present but not salient at the moment of action; and under adversarial pressure, which we do not test.

This squares with the two opposite-looking priors of Section 2. Mechanical Enforcement [arXiv:2605.14744] finds enforcement helping, but in a different domain: it scores decision-rationale quality under regulatory pressure, not tool-action compliance under benign use. Mind the GAP [arXiv:2602.16943] finds a runtime contract with no deterrent, but it measures a different quantity than we do. Its Enforce mode hard-blocks, as our commit gate does; its null is on the model's attempt rate, scored on intent before the block, under multi-turn jailbreak, with the told and enforced rules being separate objects; and it is a power-limited null ($p > 0.27$), not a demonstrated equivalence. We measure first-action compliance with the same rule told or withheld under benign pressure. Neither result contradicts ours: they score different things under different pressure.

9 Conclusion and future work

On the commit boundary, in a small pre-registered study on one runtime, telling the rule is enough for a capable model and not for a weak one. Told the rule with enforcement off, both capable models refused on every run of both tasks and both weak models committed the forbidden change on every run; because even the capable models violate when the rule is withheld, this is genuine substitution rather than a model that never needed either lever. We report it as raw counts on a two-task-per-side unit, not a significance claim, and it is confounded: capability with model family, the load-bearing arm effectively one model. The medical-boundary rule was uninformative at this scale. Enforcement is redundant only in that narrow, high-capability, benign sense; its guarantee is intact and it stays load-bearing everywhere a model cannot self-enforce.

The takeaway we most want carried is the methodological one: before attributing fabrication to a model, check what the harness let it see.

Four things would sharpen this. A powered version needs more test tasks per rule, not more repeats of the same two: the honest sample size is set by tasks, and a fresh pre-registered run with more scenarios (and a non-Qwen weak model, to break the family confound) is the legitimate route to a significance claim. External replication on a second runtime with its own rules and scorer is what would turn a case study into a phenomenon. Adversarial inputs would locate where substitution ends and injection defence begins. And long-horizon drift (does enforcement catch a rule the model has since forgotten?) is the measurement this design most directly points at.

References

- IFEval. arXiv:2311.07911.
- ST-WebAgentBench. arXiv:2410.06703.
- Agent-SafetyBench. arXiv:2412.14470.
- AgentSpec. arXiv:2503.18666.
- CaMeL: Defeating Prompt Injections by Design. arXiv:2503.18813.
- Progent. arXiv:2504.11703.
- RECAST. arXiv:2505.19030.
- VerIF. arXiv:2506.09942.
- Policy-as-Prompt. arXiv:2509.23994.
- LogiSafetyBench. arXiv:2601.08196.
- PCAS: Policy Compiler for Secure Agentic Systems (Palumbo, Choudhary, Choi, Chalasani, Jha). arXiv:2602.16708.
- Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943.
- Agent Behavioral Contracts. arXiv:2602.22302.
- ContextCov. arXiv:2603.00822.
- Verifier Tax. arXiv:2603.19328.
- NLAH. arXiv:2603.25723.
- Meta-Harness. arXiv:2603.28052.
- PhantomPolicy: Policy-Invisible Violations in LLM-Based Agents. arXiv:2604.12177.
- Symbolic Guardrails. arXiv:2604.15579.
- AHE. arXiv:2604.25850.
- TeamBench: Evaluating Agent Coordination under Enforced Role Separation. arXiv:2605.07073.
- Mechanical Enforcement. arXiv:2605.14744.
- Prompts Don't Protect. arXiv:2605.18414.
- Harness-Bench. arXiv:2605.27922.
- AIRGuard. arXiv:2605.28914.
- SABER. arXiv:2606.01317.
- What Benchmarks Don't Measure: The Case for Evaluating Abstention Competence in Autonomous Agents (Ojewale & Venkatasubramanian; ABSTAIN). arXiv:2606.02965.
- TRACE. arXiv:2606.13174.
- Constraint-driven fabrication under adversarial pressure. arXiv:2606.14831.

- Harness-MU. arXiv:2606.21856.
- Governance Decay (mitigation: Constraint Pinning). arXiv:2606.22528.
- SafePyramid. arXiv:2606.29887.
- Agent Harness Engineering: A Survey (proposes the ETCLOVG taxonomy; under review at TMLR). OpenReview 3hXEPbG0dh.

Appendix A: reproduction

The headline table reproduces from the released analysis (`per_gate_substitution.json`) and the pooling module (`build_cell_contrasts_pooled`); the exact tests beside it come from a small released helper (`exact_tests.py`, pure Python, no dependencies), which returns the run-level (0.00016), task-level (0.33), per-sub-gate (0.029), and family-corrected (0.00062) figures and the Clopper-Pearson intervals ([63, 100] for 8/8, [0, 37] for 0/8). Offline artefacts regenerate with `reproduce_offline.py` (no network, no private data). The model-backed trajectories and scores ship as a versioned archive alongside the preprint; reproducing the runtime means checking out git 6c82cd0, not installing the package. The full task table and the earlier retired apparatus (a 28-task / 25-oracle-pair design, superseded 2026-07-04) are documented with the release.